

NAME:P.VARSHITHA

BATCH:06

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
    pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

PROMPT:

generate a Stack class with push, pop, peek, and is_empty methods.

CODE:

```
11_1.py > Stack > is_empty
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        raise IndexError("pop from empty stack")

    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        raise IndexError("peek from empty stack")

    def is_empty(self):
        return len(self.items) == 0

# Example usage:
if __name__ == "__main__":
    s = Stack()
    s.push(10)
    s.push(20)
    print(s.peek())    # Output: 20
    print(s.pop())     # Output: 20
    print(s.is_empty())# Output: False
```

OUTPUT:

```
20
20
False
```

OBSERVATION:

Uses a list to store items privately (self._items) Supports standard stack operations: push, pop, peek, and is_empty Checks for empty stack before popping or peeking Returns None if operations are attempted on an empty stack avoids crashes Pushes three values: 10, 20, 30 Shows top element before and after popping Gracefully handles popping from an empty

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

PROMPT:

implement a Queue using Python lists.

CODE:

```

class Queue:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return len(self.items) == 0

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if self.is_empty():
            raise IndexError("dequeue from empty queue")
        return self.items.pop(0)

    def peek(self):
        if self.is_empty():
            raise IndexError("peek from empty queue")
        return self.items[0]

    def size(self):
        return len(self.items)

# Example usage:
if __name__ == "__main__":
    q = Queue()
    print("Is queue empty?", q.is_empty())
    q.enqueue(10)
    q.enqueue(20)
    q.enqueue(30)
    print("Queue size:", q.size())
    print("Front element:", q.peek())
    print("Dequeued:", q.dequeue())
    print("Queue size after dequeue:", q.size())
    print("Is queue empty?", q.is_empty())

```

OUTPUT:

```
Queue size: 3
Front element: 10
Dequeued: 10
Queue size after dequeue: 2
Is queue empty? False
```

OBSERVATION:

Uses a list to store items privately (self._items) Supports standard stack operations: push, pop, peek, and is_empty Checks for empty stack before popping or peeking Returns None if operations are attempted on an empty stack avoids crashes Pushes three values: 10, 20, 30 Shows top element before and after popping Gracefully handles popping from an empty

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
    pass
```

```
class LinkedList:
    pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

PROMPT:

to generate a Singly Linked List with insert and display methods.

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node

    def display(self):
        current = self.head
        while current:
            print(current.data, end=" -> " if current.next else "")
            current = current.next
        print()

# Example usage:
ll = LinkedList()
ll.insert(10)
ll.insert(20)
ll.insert(30)
ll.display() # Output: 10 -> 20 -> 30
```

OUTPUT:

```
10 -> 20 -> 30
```

OBSERVATION:

This Python code defines a basic singly linked list using a `LinkedList` class with an `insert` method that adds new nodes to the front of the list, and a `display` method to print the list elements. It initializes with an empty head and builds the list in reverse order of insertion, demonstrating how linked lists store data sequentially through node references. The sample usage shows the list being populated with values 30, 20, and 10, resulting in an output of 10 -> 20 -> 30, confirming correct insertion and traversal logic. It's a clean and effective introduction to linked list fundamentals.

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST:
```

```
    pass
```

Expected Output:

- BST implementation with recursive insert and traversal methods.

PROMPT:

create a BST with insert and in-order traversal methods.

CODE:

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        idx = self._hash(key)
        for i, (k, v) in enumerate(self.table[idx]):
            if k == key:
                self.table[idx][i] = (key, value)
                return
        self.table[idx].append((key, value))

    def search(self, key):
        idx = self._hash(key)
        for k, v in self.table[idx]:
            if k == key:
                return v
        return None

    def delete(self, key):
        idx = self._hash(key)
        for i, (k, v) in enumerate(self.table[idx]):
            if k == key:
                del self.table[idx][i]
                return True
        return False

# Example usage:
if __name__ == "__main__":
    ht = HashTable()
    ht.insert("apple", 10)
    ht.insert("banana", 20)
    print("Search apple:" ht.search("apple")) # Out
```

```
# Example usage:
if __name__ == "__main__":
    ht = HashTable()
    ht.insert("apple", 10)
    ht.insert("banana", 20)
    print("Search apple:", ht.search("apple"))    # Output: 10
    print("Search banana:", ht.search("banana"))  # Output: 20
    print("Delete apple:", ht.delete("apple"))    # Output: True

    print("Search apple:", ht.search("apple"))    # Output: None
```

OUTPUT:

```
Queue size after dequeue: 2 ...
Search banana: 20
Delete apple: True
Search apple: None
```

OBSERVATION:

This Python code defines a Binary Search Tree (BST) with methods for inserting nodes and performing in-order traversal. It uses recursion to maintain BST properties, placing smaller values to the left and larger ones to the right. The in-order traversal prints elements in sorted order, confirming correct structure. Sample usage inserts multiple values and demonstrates traversal, making it a solid example of tree-based data organization and recursive logic.

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

```
    pass
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

PROMPT:

implement a hash table with basic insert, search, and delete methods.

CODE:

```

class Graph:
    def __init__(self):
        self.adj_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []

    def add_edge(self, src, dest):
        if src not in self.adj_list:
            self.add_vertex(src)
        if dest not in self.adj_list:
            self.add_vertex(dest)
        self.adj_list[src].append(dest)

    def get_vertices(self):
        return list(self.adj_list.keys())

    def get_edges(self, vertex):
        return self.adj_list.get(vertex, [])

# Example usage:
if __name__ == "__main__":
    g = Graph()
    g.add_edge("A", "B")
    g.add_edge("A", "C")
    g.add_edge("B", "C")
    g.add_vertex("D")
    print("Vertices:", g.get_vertices())      # Output: ['A', 'B', 'C', 'D']
    print("Edges from A:", g.get_edges("A")) # Output: ['B', 'C']

```

OUTPUT:

```

Vertices: ['A', 'B', 'C', 'D']
Edges from A: ['B', 'C']

```

OBSERVATION:

This Python code implements a basic hash table using a list of buckets and a simple modulo-based hash function. It supports key operations like insertion, search, update, deletion, and display, with collision handling via chaining (lists at each index). The sample usage shows how duplicate keys are updated, how missing keys are handled gracefully, and how the table's contents can be printed clearly. Overall, it's a solid and practical example of hash table mechanics, ideal for learning or lightweight applications.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
```

```
    pass
```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

PROMPT:

implement a graph using an adjacency list.

CODE:

```
import heapq

class PriorityQueue:
    def __init__(self):
        self._heap = []

    def push(self, item, priority):
        heapq.heappush(self._heap, (priority, item))

    def pop(self):
        if self._heap:
            return heapq.heappop(self._heap)[1]
        raise IndexError("pop from an empty priority queue")

    def peek(self):
        if self._heap:
            return self._heap[0][1]
        raise IndexError("peek from an empty priority queue")

    def is_empty(self):
        return len(self._heap) == 0

    def __len__(self):
        return len(self._heap)

# Example usage:
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.push("task1", priority=3)
    pq.push("task2", priority=1)
    pq.push("task3", priority=2)

    print("Peek:", pq.peek()) # Should print 'task2'
    while not pq.is_empty():
        print("Pop:", pq.pop())
```

OUTPUT:

```
Peek: task2
Pop: task2
Pop: task3
Pop: task1
```

OBSERVATION:

This code builds an undirected graph using an adjacency list. It supports adding vertices and edges, and displays connections clearly. Sample usage shows a small network with nodes A–D and their links, making it a simple and effective intro to graph structures.

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

```
class PriorityQueue:
    pass
```

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

PROMPT:

implement a priority queue using Python's heapq module.

CODE:

```

import heapq

class PriorityQueue:
    def __init__(self):
        self._heap = []

    def push(self, item, priority):
        heapq.heappush(self._heap, (priority, item))

    def pop(self):
        if self._heap:
            return heapq.heappop(self._heap)[1]
        raise IndexError("pop from an empty priority queue")

    def peek(self):
        if self._heap:
            return self._heap[0][1]
        raise IndexError("peek from an empty priority queue")

    def is_empty(self):
        return len(self._heap) == 0

    def __len__(self):
        return len(self._heap)

# Example usage:
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.push("task1", priority=3)
    pq.push("task2", priority=1)
    pq.push("task3", priority=2)

    print("Peek:", pq.peek()) # Should print 'task2'
    while not pq.is_empty():
        print("Pop:", pq.pop())

```

OUTPUT:

```
Peek: task2
Pop: task2
Pop: task3
Pop: task1
```

OBSERVATION:

This Python code implements a priority queue using the `heapq` module, storing items as (priority, value) tuples to maintain order efficiently. It includes methods to push, pop, peek, and check if the queue is empty, with sample usage showing tasks being added and retrieved based on priority. The queue handles empty states gracefully and demonstrates correct behavior, making it a clear and practical example of heap-based priority management.

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`.

Sample Input Code:

```
class DequeDS:
    pass
```

Expected Output:

- Insert and remove from both ends with docstrings.

PROMPT:

Implement a Python class `DequeDS` using `collections.deque` to support inserting and removing elements from both front and rear. Add clear docstrings for the class and methods, and include a short demo showing how to use the deque for different operations.

CODE:

```

from collections import deque

class DequeDS:
    """
    Double-ended queue (Deque) implementation using collections.deque.
    Supports insertion and removal of elements from both front and rear.
    """

    def __init__(self):
        """Initialize an empty deque."""
        self.deque = deque()

    def add_front(self, item):
        """Insert an item at the front of the deque."""
        self.deque.appendleft(item)

    def add_rear(self, item):
        """Insert an item at the rear of the deque."""
        self.deque.append(item)

    def remove_front(self):
        """Remove and return the item from the front of the deque."""
        if self.is_empty():
            raise IndexError("remove_front from empty deque")
        return self.deque.popleft()

    def remove_rear(self):
        """Remove and return the item from the rear of the deque."""
        if self.is_empty():
            raise IndexError("remove_rear from empty deque")
        return self.deque.pop()

    def is_empty(self):
        """Check if the deque is empty."""
        return len(self.deque) == 0

```

```

def __str__(self):
    """Return a string representation of the deque."""
    return str(list(self.deque))

# Demo usage
if __name__ == "__main__":
    dq = DequeDS()
    dq.add_rear(10)
    dq.add_front(20)
    dq.add_rear(30)
    print("Deque after additions:", dq)
    print("Removed from front:", dq.remove_front())
    print("Removed from rear:", dq.remove_rear())
    print("Deque now:", dq)

```

OUTPUT:

```

Deque after additions: [20, 10, 30]
Removed from front: 20
Removed from rear: 30
Deque now: [10]

```

OBSERVATION:

This code wraps Python's deque to create a double-ended queue with methods to append, pop, peek, and display from both ends. It handles empty checks gracefully and shows how elements can be added or removed from either side. The sample usage confirms correct behavior and makes it a clear, flexible structure for queue-like tasks.

Task Description #9 – AI-Generated Data Structure Comparisons

Task: Use AI to generate a comparison table of different data structures (stack, queue, linked list, etc.) including time complexities.

Sample Input Code:

No code, prompt AI for a data structure comparison table

Expected Output:

- A markdown table with structure names, operations, and complexities.

PROMPT:

Generate a markdown table comparing common data structures (Stack, Queue, Linked List, Binary Search Tree, Hash Table, Graph, Deque, Priority Queue). Include their basic operations (insertion, deletion, search, access) and their time complexities in Big-O notation.

CODE:

```

data = [
    ["Stack", "O(1)", "O(1)", "O(n)", "O(n)"],
    ["Queue", "O(1)", "O(1)", "O(n)", "O(n)"],
    ["Linked List", "O(1)", "O(1)", "O(n)", "O(n)"],
    ["Binary Search Tree", "O(log n)", "O(log n)", "O(log n)", "O(log n)"],
    ["Hash Table", "O(1)", "O(1)", "O(1)", "O(1)"],
    ["Graph (Adj. List)", "O(1)", "O(1)", "O(V+E)", "O(1)"],
    ["Deque", "O(1)", "O(1)", "O(n)", "O(n)"],
    ["Priority Queue", "O(log n)", "O(log n)", "O(n)", "O(1)"]
]

header = ["Data Structure", "Insertion", "Deletion", "Search", "Access"]

# Print markdown table
print("| " + " | ".join(header) + " |")
print("|" + "|".join(["-" * len(h) for h in header]) + "|")
for row in data:
    print("| " + " | ".join(row) + " |")

print("\n* O(1) if inserting/deleting at the head (singly linked list).")
print("** O(log n) for balanced BSTs; O(n) for unbalanced BSTs.")
print("V = number of vertices, E = number of edges.")

```

OUTPUT:

```

| Binary Search Tree | O(log n) | O(log n) | O(log n) | O(log n) |
| Hash Table | O(1) | O(1) | O(1) | O(1) |
| Graph (Adj. List) | O(1) | O(1) | O(V+E) | O(1) |
| Deque | O(1) | O(1) | O(n) | O(n) |
| Priority Queue | O(log n) | O(log n) | O(n) | O(1) |

* O(1) if inserting/deleting at the head (singly linked list).
** O(log n) for balanced BSTs; O(n) for unbalanced BSTs.
V = number of vertices, E = number of edges.

```

OBSERVATION:

This code models a real-world cafeteria order system using Python's deque for efficient queue management. It supports adding orders, serving them in FIFO order, and displaying the current queue. The sample usage shows how orders are processed and updated, making it a practical and intuitive example of applying data structures to everyday scenarios.

Task Description #10 Real-Time Application Challenge – Choose the Right Data Structure Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - Stack
 - Queue
 - Priority Queue
 - Linked List
 - Binary Search Tree (BST)
 - Graph
 - Hash Table
 - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

PROMPT:

This table compares data structures by operation speed. Arrays are fast for access, slow for changes. Linked lists and queues are quick to insert/delete but slow to search. Hash tables are fastest overall (average $O(1)$). Trees and heaps offer balanced performance, ideal for sorted or priority data.

CODE:

```

from collections import deque

class CafeteriaOrderQueue:
    def __init__(self):
        self.queue = deque()

    def add_order(self, student_name, order):
        self.queue.append((student_name, order))

    def serve_order(self):
        if self.queue:
            student_name, order = self.queue.popleft()
            print(f"Serving {student_name}: {order}")
        else:
            print("No orders to serve.")

    def show_queue(self):
        if not self.queue:
            print("The order queue is empty.")
        else:
            print("Current order queue:")
            for idx, (student_name, order) in enumerate(self.queue, 1):
                print(f"{idx}. {student_name}: {order}")

# Example usage
cafeteria = CafeteriaOrderQueue()
cafeteria.add_order("Alice", "Veg Sandwich")
cafeteria.add_order("Bob", "Pasta")
cafeteria.add_order("Charlie", "Juice")
cafeteria.show_queue()
cafeteria.serve_order()
cafeteria.show_queue()

```

OUTPUT:

```
Current order queue:
1. Alice: Veg Sandwich
2. Bob: Pasta
3. Charlie: Juice
Serving Alice: Veg Sandwich
Current order queue:
1. Bob: Pasta
2. Charlie: Juice
```

OBSERVATION:

This code models a real-world cafeteria order system using Python's deque for efficient queue management. It supports adding orders, serving them in FIFO order, and displaying the current queue. The sample usage shows how orders are processed and updated, making it a practical and intuitive example of applying data structures to everyday scenarios.